SIGELAB móvil

Guía para poner a funcionar la app móvil, la API y la conexión con la laptop después de descargar el proyecto del repositorio

Proyecto Final · Programación II · UMG | Windows 10 / 11 | Tiempo aproximado: 1 a 2 horas la primera vez

Respuesta corta: no hay que cambiar código
Todo lo que cambia de una computadora a otra (contraseña de la base de datos, dirección del servidor, puerto) se configura con variables de Windows o desde la misma app. La sección 9 explica qué tocar si su caso es distinto.

Contenido

1. Cómo funciona (sin tecnicismos)
2. Qué hay que instalar
3. Descargar el proyecto
4. Base de datos en Docker
5. Encender la API
6. Flutter y Android
7. Ver la app en el emulador
8. Generar la APK y usarla en un teléfono
9. ¿Hay que cambiar algo del código?
10. Problemas comunes y su solución
11. Resumen: cada vez que lo usen

1. Cómo funciona (sin tecnicismos)

Piense en la laptop como una **oficina** y en el teléfono como alguien que **llama** a esa oficina para hacer preguntas.

Pieza	Es como…	Qué hace
Docker con SQL Server	El archivo	Guarda toda la información: equipos, préstamos, usuarios.
API (carpeta `api`)	La recepcionista	Atiende al teléfono, revisa usuario y contraseña, busca en el archivo y responde.
Túnel (Cloudflare)	Un número público	Le da a la laptop una dirección en internet para que el teléfono la encuentre desde cualquier lugar.
App móvil (carpeta `movil`)	Quien llama	Pide datos y los muestra. Nunca entra directo al archivo.
App de escritorio (carpeta `src`)	Un empleado de la oficina	Registra préstamos, devoluciones, etc. directo en el archivo.



El teléfono solo habla con la API. Lo que se registra en el escritorio aparece en el teléfono al deslizar la pantalla hacia abajo.

Cada laptop tiene su propia base de datos

Quien descargue el proyecto tendrá una **SigelabDB nueva**, con los datos de prueba, no con los datos de otra persona. El teléfono verá los datos de la laptop a la que se conecte. Si necesitan exactamente los mismos datos, el dueño debe pasarles un respaldo de su base.

2. Qué hay que instalar

Necesita una laptop con Windows 10/11, 16 GB de RAM recomendados (8 GB funciona sin emulador) y unos 20 GB libres. La forma más fácil de instalar todo es con **winget**, que ya viene en Windows. Abra **PowerShell** y copie cada línea:

Programa	¿Para qué?	Comando para instalarlo
Git	Descargar el repositorio	`winget install Git.Git`
Docker Desktop	La base de datos	`winget install Docker.DockerDesktop`
JDK 21 (Eclipse Temurin)	Ejecutar la API	`winget install EclipseAdoptium.Temurin.21.JDK`
Android Studio	SDK de Android y emulador	`winget install Google.AndroidStudio`
VS Code	Abrir y ejecutar la app	`winget install Microsoft.VisualStudioCode`
cloudflared	El túnel a internet	`winget install --id Cloudflare.cloudflared`
PowerShell 7 (recomendado)	Ejecutar los scripts	`winget install Microsoft.PowerShell`
Flutter	Compilar la app	No está en winget: se instala en la **sección 6**.

Use un JDK normal, no GraalVM

Con el JDK de GraalVM la compilación de Android falla (error `Module jdk.internal.vm.ci not found`). Temurin (el del comando de arriba) o el que trae Android Studio funcionan bien.

Después de instalar
Cierre y vuelva a abrir la terminal para que Windows reconozca los programas nuevos. Docker Desktop pide reiniciar la computadora la primera vez y debe quedar abierto (ícono de la ballena en la barra de tareas).

3. Descargar el proyecto

```
mkdir C:\dev
cd C:\dev
git clone https://github.com/SaideVG/ProyectoLab_Prograll.git
cd ProyectoLab_Prograll
```

No lo descargue dentro de OneDrive (Documentos, Escritorio)
OneDrive bloquea los miles de archivos temporales que crea Android al compilar y aparece el error `Unable to delete directory ... mergeDebugAssets`. Use una carpeta como `C:\dev`. Descárguelo con `git clone` y no como ZIP: Windows marca los ZIP de internet y luego bloquea los scripts.

Lo que va a encontrar:

Carpeta	Contenido
`database\`	Scripts que crean la base **SigelabDB** con datos de prueba.
`src\` y `pom.xml`	La app de escritorio (se abre en IntelliJ; no hace falta para la parte móvil).
`api\`	La API en Spring Boot y los scripts `iniciar_api.ps1` e `iniciar_tunel.ps1`.
`movil\`	La app de Flutter. El código está en `movil\lib\`.

4. Base de datos en Docker

Con Docker Desktop abierto, cree el contenedor de SQL Server **una sola vez**. La contraseña `ProyecLAB26` es la que el proyecto usa por defecto; si usa esa, no tiene que configurar nada más:

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=ProyecLAB26" -p 1433:1433 --name sqlserver
-d mcr.microsoft.com/mssql/server:2022-latest
```

Espere unos 20 segundos y cree la base con los datos de prueba (desde la carpeta del proyecto):

```
.\database\cargar_en_docker.ps1
```

Debe terminar con `SigelabDB lista en localhost:1433.` Los usuarios de prueba que crea son:

Usuario	Contraseña	Rol
`admin`	`Admin2026!`	Administrador
`encargado`	`Encargado2026!`	Encargado
`docente`	`Docente2026!`	Docente
`inactivo`	`Inactivo2026!`	No puede entrar (usuario inactivo, a propósito)

> **¿Ya tenía SQL Server con otra contraseña?**
> No cambie el código. Guarde la contraseña en una variable de Windows y abra una terminal nueva: `setx DB_PASSWORD "suContraseña"`. La API y el escritorio la leen de ahí.
> **Ojo:** `admin` / `admin` solo funciona en el escritorio (es un atajo de prueba). En el teléfono use `Admin2026!`.

En los días siguientes no hay que repetir esto: basta con abrir Docker Desktop (o ejecutar `docker start sqlserver`).

5. Encender la API

En una terminal, dentro de la carpeta `api`:

```
cd C:\dev\ProyectoLab_Prograll\api
.\iniciar_api.ps1
```

La primera vez descarga lo que necesita (2 a 5 minutos). Está lista cuando aparece `Started SigelabApiApplication`. **Deje esa ventana abierta**; para apagarla, `Ctrl + C`.

Para comprobarla, abra en el navegador `http://localhost:8080/api/estado`. Debe mostrar `"baseDeDatos":"conectada"`.

> **Si Windows dice "la ejecución de scripts está deshabilitada"**
> Use PowerShell 7 (`pwsh`), que sí permite scripts locales, o ejecute el comando directo, que hace lo mismo sin script:
>
> ```
> .\mvnw.cmd spring-boot:run
> ```

Alternativa en IntelliJ: clic derecho en `api\pom.xml` → *Add as Maven Project*, y arriba a la derecha elegir **SIGELAB API** → ▶.

6. Flutter y Android

6.1 Instalar Flutter

```
git clone https://github.com/flutter/flutter.git -b stable C:\dev\flutter
```

Agregue Flutter al PATH:

1. Tecla Windows → escriba **variables de entorno** → *Editar las variables de entorno de esta cuenta*.
2. Seleccione **Path** → **Editar** → **Nuevo** → escriba `C:\dev\flutter\bin` → Aceptar.
3. Abra una terminal **nueva** y ejecute `flutter --version` (la primera vez descarga más archivos).

6.2 Activar el Modo de desarrollador de Windows

Configuración → Sistema → Para programadores → Modo de desarrollador: Activado. Sin esto, Flutter muestra `Building with plugins requires symlink support`.

6.3 Preparar Android Studio

1. Abra **Android Studio** y siga el asistente con la opción **Standard**: instala el SDK de Android y el emulador.
2. En la pantalla de bienvenida: *More Actions → SDK Manager → pestaña SDK Tools* → marque **Android SDK Command-line Tools** → Apply.
3. Acepte las licencias de Android (escriba `y` en cada pregunta):

   ```
   flutter doctor --android-licenses
   ```

4. Cree el emulador: *More Actions → Virtual Device Manager → Create Virtual Device* → **Pixel** → una imagen con **API 36** → Finish. En *Graphics* elija **Hardware** para que se vea fluido.

6.4 Revisar que todo esté bien

```
flutter doctor
```

Deben salir con ✓ **Flutter** y **Android toolchain**. "Visual Studio" puede salir con *X*: solo sirve para apps de Windows y no se usa.

6.5 VS Code

En VS Code: *Extensiones* (Ctrl+Shift+X) → busque **Flutter** (de Dart Code) → Instalar.

7. Ver la app en el emulador

Con **Docker** y la **API** encendidos:

1. En VS Code: *Archivo → Abrir carpeta* → elija **solo** la carpeta `movil`. Si pregunta, marque **Confiar**.
2. En la esquina **inferior derecha** haga clic donde dice *No Device* (o *Chrome*) y elija su emulador.
3. Abra `lib\main.dart` y presione **F5**. La primera compilación tarda varios minutos.
4. Entre con `admin` / `Admin2026!`.

> **En el emulador no hay que configurar el servidor**
> Ya viene con `http://10.0.2.2:8080`, que para el emulador significa "la laptop donde corro". Por terminal es lo mismo: `cd movil`, `flutter pub get` y `flutter run`.

8. Generar la APK y usarla en un teléfono

8.1 Crear la APK

```
cd C:\dev\ProyectoLab_Prograll\movil
flutter build apk --release --split-per-abi
```

Use `movil\build\app\outputs\flutter-apk\app-arm64-v8a-release.apk` (sirve para casi todos los teléfonos actuales). Pásela al teléfono por WhatsApp, Drive o cable.

8.2 Instalarla

1. Abra el archivo APK en el teléfono.
2. Android pedirá permitir *Instalar apps desconocidas* para la app con la que la abrió: permítalo.
3. Si Play Protect avisa "app no reconocida", elija **Instalar de todas formas** (es normal: no viene de Play Store).

8.3 Conectar el teléfono con la laptop

El teléfono es otro aparato: necesita un "camino" hasta la laptop. Elija uno:

Camino	En la laptop	En el teléfono, en **Servidor**
Túnel desde cualquier lugar, con Wi-Fi o datos	Con la API encendida, en otra terminal dentro de `api`: `.\iniciar_tunel.ps1` Muestra una dirección `https://….trycloudflare.com` y la copia sola. Sin script: `cloudflared tunnel --url http://localhost:8080`	La dirección del túnel. **Cambia cada vez** que se abre el túnel.
Misma Wi-Fi	Ver la IP con `ipconfig` (línea "Dirección IPv4" del adaptador Wi-Fi). La red Wi-Fi debe estar como **Privada**: *Configuración → Red e Internet → Wi-Fi → su red → Privada*.	La IP con el puerto, por ejemplo `192.168.0.8:8080`
Cable USB	Teléfono con *Depuración USB* activada, conectado por cable; ejecutar `adb reverse tcp:8080 tcp:8080` `adb` está en `%LOCALAPPDATA%\Android\Sdk\platform-tools`	`localhost:8080`

8.4 Primera vez en el teléfono

1. Abra **SIGELAB**. Abajo, el botón **Servidor** aparece resaltado y dice *Sin configurar*: tóquelo.
2. Pegue la dirección (no hace falta escribir `https://`; la app lo agrega) → **Probar** → debe decir "*Conexión correcta*" → **Guardar**.
3. Entre con su usuario. Las siguientes veces entra directo.
4. Para ver cambios hechos en el escritorio, deslice la pantalla hacia abajo.

El teléfono funciona solo mientras la laptop está lista
Docker, la API y (si usa túnel) la ventana del túnel deben estar abiertos, y la laptop no debe suspenderse (*Configuración → Sistema → Energía*). Si el túnel se reinicia, mande la dirección nueva y en el teléfono toque **Servidor** para cambiarla. iPhone no se puede: la APK es solo para Android.

9. ¿Hay que cambiar algo del código?

No para que funcione. Esta tabla dice qué hacer si su situación es distinta a la normal:

Situación	Qué hacer	¿Editar código?
Su SQL Server tiene otra contraseña	`setx DB_PASSWORD "suContraseña"` y abrir una terminal nueva	No
Otro servidor, otra base u otro usuario de SQL	`setx DB_URL "jdbc:sqlserver://localhost:1433;databaseName=SigelabDB;encrypt=true;trustServerCertificate=true;sendTimeAsDatetime=false;"` y `setx DB_USER "usuario"`	No
El puerto 8080 ya está ocupado	`setx PORT 8081` (la API y el script del túnel lo leen). En el teléfono use `:8081`. En el emulador escriba `http://10.0.2.2:8081` en Servidor.	No
Quieren que la APK ya traiga la dirección del servidor	`flutter build apk --release --split-per-abi --dart-define=SERVIDOR=https://su-direccion`	No
Cambiar intentos de login antes del bloqueo (5) o los minutos (15)	`api\src\main\resources\application.properties`: `sigelab.login.max-intentos` y `sigelab.login.minutos-bloqueo`	Configuración
Cambiar colores de la app	`movil\lib\config\tema.dart` (son los mismos de `Tema.java` del escritorio)	Sí
Agregar o cambiar una consulta	API: `repository` → `service` → `controller`. App: `services\api_service.dart` → `models\` → `views\`	Sí

No edite ni suba estos archivos
`movil\build\`, `movil\.dart_tool\`, `movil\android\local.properties` y `api\target\` se generan solos en cada computadora (Git ya los ignora). Tampoco suba contraseñas: use las variables de Windows.

Dónde está cada cosa del código

Archivo o carpeta	Qué hace
`movil\lib\main.dart`	Arranque de la app y regreso al login si la sesión vence.
`movil\lib\services\api_service.dart`	Único lugar que habla con la API (como los Repository del escritorio).
`movil\lib\services\sesion.dart`	Usuario conectado, token y dirección del servidor guardada en el teléfono.
`movil\lib\views\`	Una pantalla por archivo: login, inicio, equipos, escáner QR, préstamos, mantenimiento.
`api\...\controller\`	Las rutas que usa la app (`/api/auth/login`, `/api/equipos`, …).
`api\...\service\UsuarioService.java`	Login: BCrypt, usuario activo (RN-01), auditoría y bloqueo por intentos.
`api\README.md` y `movil\README.md`	Explicación técnica completa de cada parte.

10. Problemas comunes y su solución

Lo que aparece	Causa y solución
`flutter`, `git` o `docker` "no se reconoce como comando"	Falta el PATH o la terminal es vieja. Cierre y abra una terminal nueva; revise la sección 6.1.
"la ejecución de scripts está deshabilitada en este sistema"	Use PowerShell 7 (`pwsh`) o el comando directo de la sección 5. No descargue el proyecto como ZIP.
La API dice `Login failed for user 'sa'`	La contraseña del contenedor no es `ProyecLAB26` : `setx DB_PASSWORD "suContraseña"` y reinicie la API en una terminal nueva.
`/api/estado` dice `"sin conexión"`	Docker Desktop cerrado o contenedor apagado: `docker start sqlserver` .
`Unable to delete directory ... mergeDebugAssets`	El proyecto está dentro de OneDrive. Vuelva a clonarlo en `C:\dev` .
`Module jdk.internal.vm.ci not found` o falla `jlink`	Flutter está usando GraalVM. Indíquele otro JDK: `flutter config --jdk-dir "C:\Program Files\Eclipse Adoptium\jdk-21…"` (la carpeta exacta que tenga).
`Building with plugins requires symlink support`	Active el Modo de desarrollador (sección 6.2).
`Android license status unknown`	`flutter doctor --android-licenses` y responda `y` .
En la app: *No se pudo conectar…*	API o túnel apagados, o cambió la dirección del túnel. Toque **Servidor** y póngala de nuevo.
Usuario o contraseña incorrectos con `admin/admin`	Ese atajo solo existe en el escritorio. Use `Admin2026!` .
Demasiados intentos fallidos…	5 contraseñas malas seguidas: espere 15 minutos o reinicie la API.
Su sesión terminó	Se reinició la API. Vuelva a iniciar sesión.
El emulador va lento	Virtual Device Manager → editar el dispositivo → *Graphics: Hardware*. Para ver la velocidad real: `flutter run --profile` .
La cámara no lee el QR en el emulador	El emulador usa una cámara simulada. Escriba el código abajo (ej. `CAM-001`) o pruebe en un teléfono real.

11. Resumen: cada vez que lo usen

En la laptop (en este orden)

1. Abrir **Docker Desktop** (o `docker start sqlserver`).
2. Terminal 1, en `api`: `.\iniciar_api.ps1` → esperar *Started SigelabApiApplication*.
3. Solo si el teléfono está fuera de la red: terminal 2, en `api`: `.\iniciar_tunel.ps1` → enviar la dirección.
4. Opcional: abrir la app de escritorio para registrar préstamos, devoluciones, etc.

En el teléfono

1. Abrir SIGELAB. Si cambió la dirección del túnel: **Servidor** → pegar → **Guardar**.
2. Entrar y deslizar hacia abajo para ver lo más reciente.

Antes de subir el proyecto al repositorio (para el dueño)

- Incluir en el commit las carpetas `api\`, `movil\` y el archivo `.run\SIGELAB API.run.xml`.
- **No** incluir `exportaciones\` (tiene correos y nombres de usuarios) ni las imágenes de `qr\`.
- Subir con IntelliJ: *Git* → *Commit* y luego *Git* → *Push* (`Ctrl+Shift+K`).